

Reviewer Pack — Minimal Index of Quaternion Algebras and SAT Clause Entropy

Entry 8cd9aa4e1e6b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 18:35:58 UTC

Minimal Index of Quaternion Algebras and SAT Clause Entropy

Entry ID: 8cd9aa4e1e6b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 18:35:58 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternionic Algebras) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Entropy)

Statement:

For every boolean formula in conjunctive normal form (CNF) with n clauses, the minimal index of a quaternion algebra that can represent all possible truth assignments of the CNF is polynomially related to the entropy of the clause set of the CNF.

Rationale (proposer's reasoning):

Quaternionic algebras provide a non-commutative generalization of complex numbers and have been studied in various contexts. Their minimal index could potentially reveal structural information about the complexity of satisfiability problems, particularly in terms of clause set randomness. This bridge could expose new insights into the relationship between algebraic structures and computational complexity.

Taxonomy category: QuaternionicAlgebra (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eb2bea45278cc121

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal index of quaternion algebras and clause entropy exceeds 0.7 for at least 80% of the CNFs tested, with a p -value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Quaternionic Algebras' AND 'Boolean Satisfiability' - 'CNF minimal index' AND 'quaternion algebra representation' - 'SAT clause entropy' related to 'minimal index of quaternion algebras'

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/1710.01374v2] Free-Boolean independence for pairs of algebras - [http://arxiv.org/abs/2412.13827v1] Bounds for the Zeros of Quaternionic Polynomials and Regular Functions Using Matrix Techniques - [http://arxiv.org/abs/1205.6564v5] Classifying complements for Hopf algebras and Lie algebras - [http://arxiv.org/abs/1812.03586v4] Symmetric Decomposition of the Associated Graded Algebra of an Artinian Gorenstein Algebra - [http://arxiv.org/abs/2101.11121v3] Holomorphic representation of minimal surfaces in simply isotropic space - [http://arxiv.org/abs/1908.01624v1] Learned Clause Minimization in Parallel SAT Solvers - [http://arxiv.org/abs/2207.13577v2] Scalable Proof Producing Multi-Threaded SAT Solving with Gimsatul through Sharing instead of Copying Clauses

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A
```

```

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0 for _ in range(p)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][k] += A[i][j] * B[j][k]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1) ** j * A[0][j] * determinant(submatrix)
    return det

def minimal_index_of_quaternion_algebra(n):
    # Placeholder function to calculate the minimal index of quaternion algebra
    # This is a dummy implementation and should be replaced with actual logic
    return n**2 # Example: minimal index is proportional to n^2

def entropy(clause_set):
    counts = [clause_set.count(c) for c in set(clause_set)]
    probabilities = [c / len(clause_set) for c in counts]
    return -sum(p * math.log2(p) for p in probabilities if p > 0)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    clause_set = ['c' + str(i) for i in range(n)]
    random.shuffle(clause_set)
    minimal_index = minimal_index_of_quaternion_algebra(n)
    clause_entropy = entropy(clause_set)
    results.append({
        "n": n,
        "minimal_index": minimal_index,
        "clause_entropy": clause_entropy
    })

metric_value = sum(result["minimal_index"] * result["clause_entropy"] for result in results) /
instances_tested = len(results)
n_max = max(result["n"] for result in results)
conjecture_holds = False # Placeholder, replace with actual logic
counterexample = ""

return {
    "metric_name": "Minimal Index * Clause Entropy",
    "metric_value": metric_value,
    "instances_tested": instances_tested,

```

```

        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

re_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}
TRIAL: {'metric_name': 'Minimal Index * Clause Entropy', 'metric_value': 2654.8915202825383, 'instance': 'Minimal Index * Clause Entropy'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a8284110.py", line 108, in <module>
    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
                        ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a proper evaluation of the conjecture. | next: Investigate and fix the crash in the test code to allow for a valid assessment of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14030
2	propose	ollama_remote	glm4:latest	0	0	12150
3	preregistration	ollama_remote	glm4:latest	0	0	9700
4	novelty	ollama_remote	glm4:latest	0	0	12154
5	novelty	ollama_remote	glm4:latest	0	0	13528
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14447
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11595
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12280
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12429
10	judge	ollama_remote	glm4:latest	0	0	13140

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125453 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/8cd9aa4e1e6b.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8cd9aa4e1e6b.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8cd9aa4e1e6b.tar.gz (if generated)